

Tidal Sensor

A low-cost, open-source ultrasonic water-level sensor deployed at Stuy Cove Park to monitor tidal changes and splash over events at the seawall.

This open-source sensor combines ultrasonic ranging, a microcontroller, and cellular connectivity to continuously measure and transmit water level data. The design originates from COAST Lab's Open Water Level initiative, which aims to democratize coastal monitoring for educational, community, and research use.

In this version, undergraduate CS interns from Oyate Group worked within our MakerLab to build and install one of these sensors at Stuy Cove Park. The device helps park staff track water level fluctuations and could provide insight into when giant waves splash over the seawall, which has been a growing problem for the park. This collaboration also supports Stuy Cove Park's broader education and tech initiatives which included environmental sensing and soil monitoring projects.



Sebastian Bidegain | Design & Tech Lead, MakerLAB

Valentina Quiroga | Intern, MakerLAB

Mera Shivlochan | Intern, MakerLAB



Candace Thompson
Manager, Stuy Cove Park



Phil Bresnahan
University of North
Carolina Wilmington

About the project

This project was started last summer as a collaboration between the MakerLAB and Stuy Cove Park, who wanted to study how tides and boat waves fluctuate daily and seasonally. Using a public Github repository that described the steps to create a cost-friendly water level sensor, we were able to create just that.



Public Github Repository



Throughout this project, we followed the step-by-step instructions outlined in a public GitHub repository created by **Phil Bresnahan**, a coastal oceanographer and Associate Professor at the University of North Carolina Wilmington.

His detailed documentation and personal guidance were essential to our progress. If you're interested in learning more or exploring the build yourself, scan the QR code to view the GitHub repository.



Process Diagram

Firmware

Downloading Particle Workbench to flash code onto Boron device

Assembly

Following the steps found on the Github repository to assemble sensor

Retrieving Materials

Getting materials and using resin 3D printing to make chassis



Particle Integration

Setting up Google apps scripts and Particle integration to record data

Data Analysis

Analysing the data recorded and adjusting parameters for use case

Deployment

Deploying project onsite and helping install it

Retrieving Materials

In collaboration with **Stuyvesant Cove Park**, they provided most of the materials needed to build the tidal sensor based on the published Bill of Materials. The core components include the Particle Boron microcontroller, ultrasonic distance sensor, and the solar power setup, along with the PVC enclosure and waterproofing supplies. For the structure, we used STL files from the GitHub repository, and resin 3D-printed the internal chassis and PVC top to securely hold and protect the sensor for outdoor deployment.

Bill of Materials – Tidal Sensor (PVC Design)

Core Electronics

- Particle Boron LTE CAT-M1 (w/ antenna)
- MaxSonar Ultrasonic Distance Sensor
- Adalogger FeatherWing (RTC + SD)
- 8GB MicroSD Card
- 3-pin Cable

Power System

- 5W / 5V Solar Panel + Battery Pack (3.7V 4400mAh)
- USB A → Micro-B Cable
- Particle Cellular Plan (Free tier)

Enclosure & Structure

- 1.5" PVC Pipe + Couplers + Reducer Bushings
- PVC Mechanical Test Plug
- O-Rings + Cable Gland
- 3D Printed Chassis (Formlabs Clear Resin)
- 3D Printed PVC Top (Tough Resin 2000)

Fasteners & Mounting

- Stainless Steel M2 Screws + Hex Nuts
- Stacking Header Pins

Sealing & Waterproofing

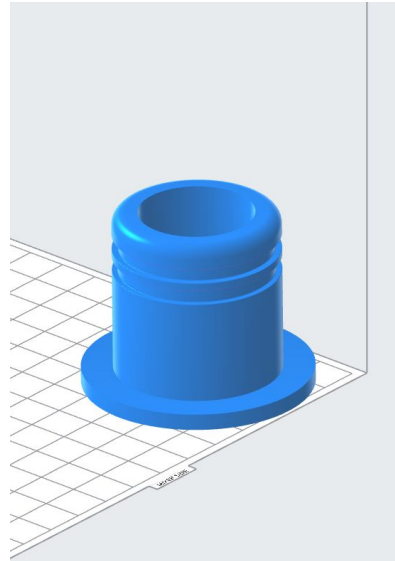
- PTFE Pipe Tape
- Silicone Grease
- Silicone Sealant
- Loctite Marine Epoxy
- PVC Cement

Estimated Cost per Unit:

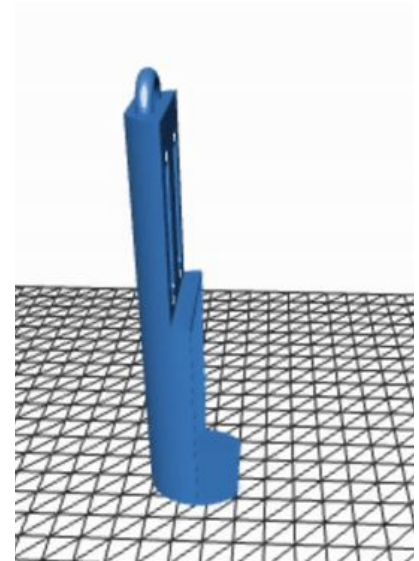
~\$311

Making the 3D printed chassis

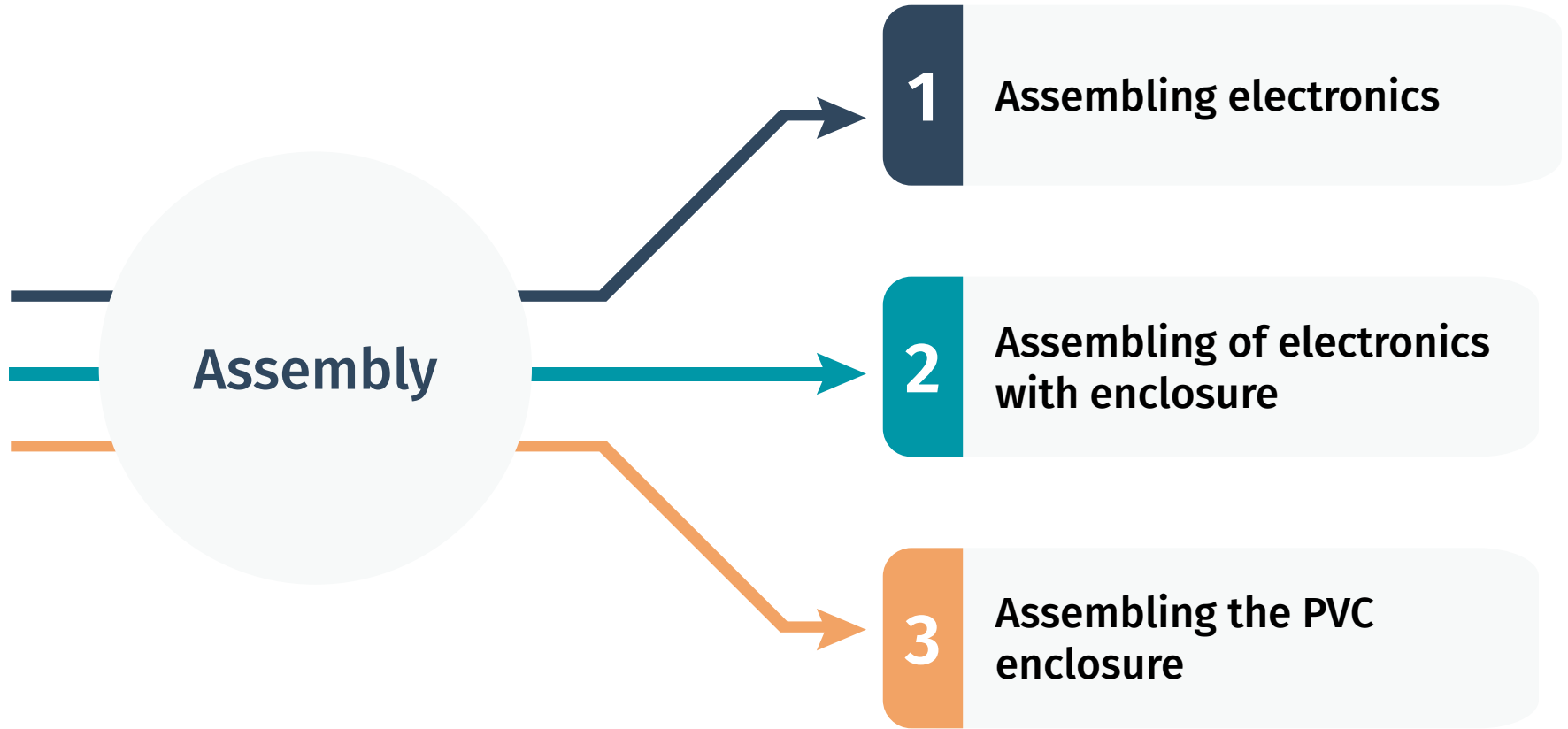
Using the resin 3D printer, Formlabs, and the slicer program PreForm, we were able to visualize and print the chassis and end cap for the PVC pipe. These files were provided through the GitHub repository.



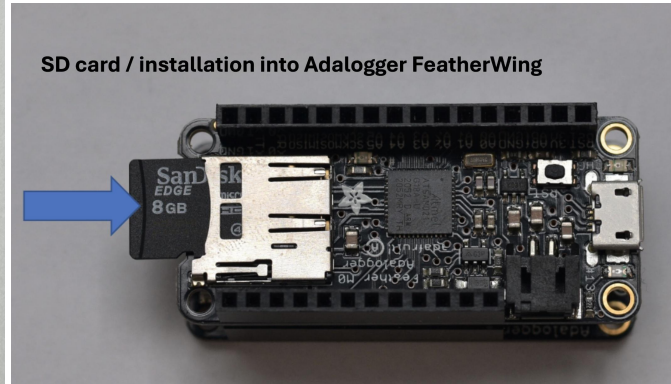
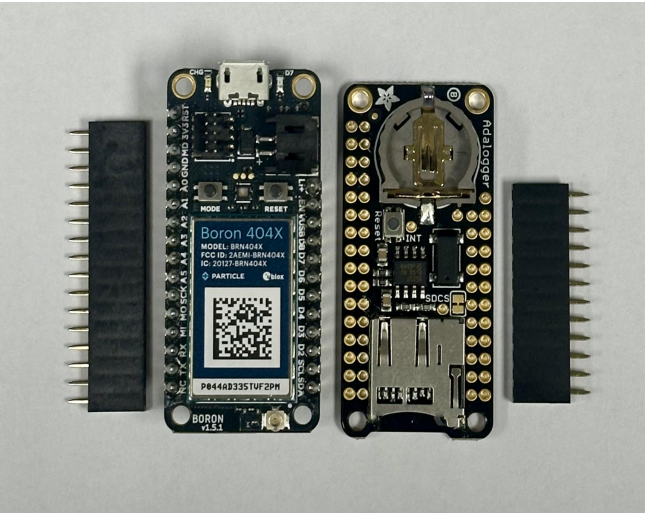
PVC End Cap



PVC Chassis

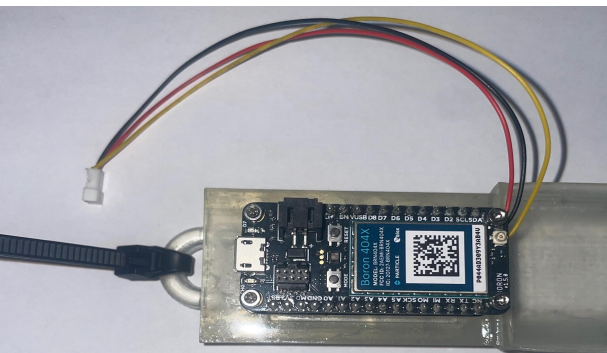


Assembling electronics



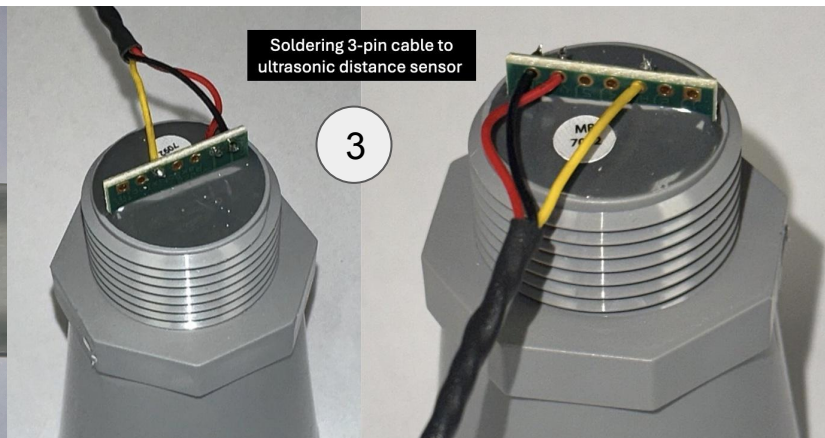
Our first step was to solder the Boron 404X on top of the Adalogger FeatherWing with stacking header pins in between them. This is in preparation for the firmware update, and so the SD card could store the data

Assembling of electronics with enclosure

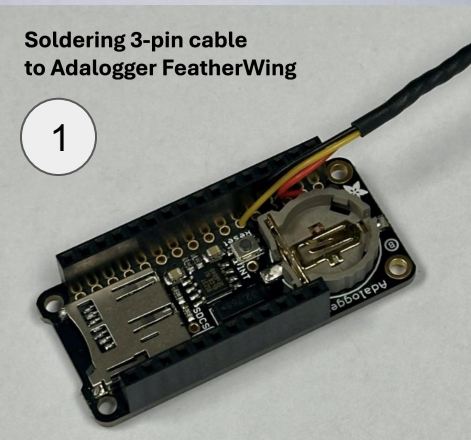


Using the Screws + Hex Nuts attach the Boron to the chassis

2

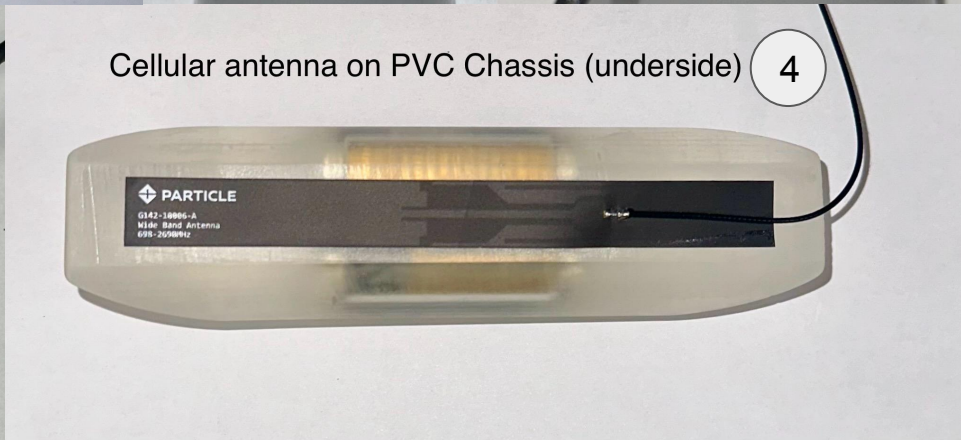


3

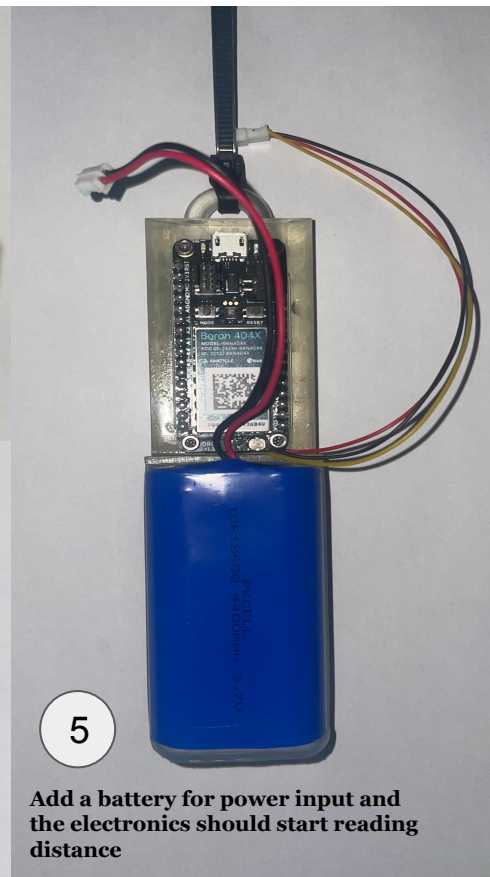


Soldering 3-pin cable to Adalogger FeatherWing

1



4



5

Add a battery for power input and the electronics should start reading distance

Assembling the PVC enclosure

After assembling the electronic components, we built the PVC enclosure and securely mounted both the sensor and the solar panel. This image shows the fully assembled system, with the microcontroller housed inside the PVC pipe.

5W / 5V
Solar panel

MaxSonar
Ultrasonic
Distance
Sensor

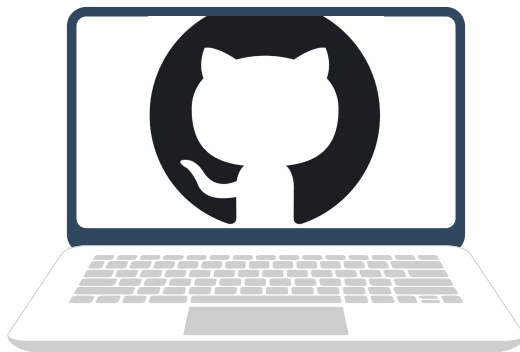


Firmware



Source

The firmware used for this project comes from the public GitHub repository, along with all the necessary libraries.



Output

The firmware instructs the device where to store its data, either on a SD card or on the Particle cloud



Firmware Overview

The screenshot displays the Particle Workbench IDE interface. The main editor window shows the source code for `SLR_Boron_Maxbotix_MB7092_cm.cpp`. The code includes headers for `FuelGauge`, `batteryMonitor`, and `SerialLogHandler`. It defines a constant `eventName` as "waterLevel" and a `logHandler`. A `SystemSleepConfiguration` object is configured. Various timing constants are defined, including `MAX_TIME_TO_PUBLISH_MS` (600000) and `TIME_AFTER_PUBLISH_MS` (4000). A comment indicates that the code is for a Boron device. The `setup` function is defined, which calls `Particle.connect()` if publishing is enabled. The bottom panel shows the `TERMINAL` tab with the output of the flashing process, including the creation of the executable file and the successful flashing of the firmware to the Boron device.

```
src > C:\SLR_Boron_Maxbotix_MB7092_cm.cpp > ...
61 FuelGauge batteryMonitor;
62 const char * eventName = "waterLevel";
63 SerialLogHandler logHandler;
64
65 SystemSleepConfiguration config;
66
67 // Various timing constants
68 const unsigned long MAX_TIME_TO_PUBLISH_MS = 600000; // Only stay awake for this time trying to connect to the cloud and publish. 11 min
69 const unsigned long TIME_AFTER_PUBLISH_MS = 4000; // After publish, wait 4 seconds for data to go out
70
71 // ***** IMPORTANT!!!
72 //If SECONDS_BETWEEN_MEASUREMENTS < 600, must use
73 //..network(NETWORK_INTERFACE_CELLULAR, SystemSleepNetworkFlag::INACTIVE_STANDBY);
74 // in sleep configuration to avoid reconnection penalty
75 const unsigned long SECONDS_BETWEEN_MEASUREMENTS = 300; // What should sampling period be?
76 // ***** IMPORTANT!!! See note above this const.
77
78
79 void setup(void) {
80   if (PUBLISHING==0) {
81     Particle.connect();
82   }
83 }
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS MEMORY XRTOS

```
*** FLASHING APPLICATION
Creating /Users/valentina/Documents/maker_lab/Tidal_Sensor_Project/Tidal_Sensor_Project/target/6.3.4/boron/platform_user_ram.ld ...
Creating /Users/valentina/Documents/maker_lab/Tidal_Sensor_Project/Tidal_Sensor_Project/target/6.3.4/boron/platform_user_ram.ld ...
text data bss dec hex filename
25652 120 2526 28298 6e8a /Users/valentina/Documents/maker_lab/Tidal_Sensor_Project/Tidal_Sensor_Project/target/6.3.4/boron/Tidal_Sen
sor_Project.elf

*** STARTING FLASH
Flashing boron e00fce68ea86a38372bc7b64
[ ] 100% | Flash success!

*** FLASHED SUCCESSFULLY ***
Press any key to close the terminal.
```

This is where the main source code is at. It was provided by the original repo (coded in c++)

Specific part that controls how often the Boron retrieves data

Compile makes an executable file to send to Boron

Flash sends the firmware to Boron, can be done remotely

Firmware

When we initially flashed the firmware onto the Boron via micro-USB, all data was stored locally on a SD card. The data was saved as a CSV file containing the unix timestamp, distance in centimeters, cell voltage, and state of charge. However, manually removing and reading the SD card was not practical for long-term deployment, so we transitioned to sending the data over cellular instead.

	A	B	C	D
	distance			
1	946684810	132.07500	0.26	91.49
2	946684810	68.24028	0.23	47.71
3	946684810	150.79306	4.17	97.98
4	946685472	42.74445	4.16	97.50
5	946686133	162.20694	4.16	97.18
6	946686793	162.07777	4.15	96.71
7	946687451	144.17500	4.15	96.39
8	946688114	136.52361	4.14	95.92
9	946688774	137.34723	4.14	95.45
10	946689434	139.35001	4.14	94.97

Particle Integration

	A	B	C	D	E	F	G
	Device ID	Date & Time	Sensor	Unix	Distance (cm)	Cell V	State of charge
2	e00fce68ea86a38372bc7b64	11/19/2025 14:45:38	Tidal_Sensor	1763581529	17.29167	4.05	79.61
3	e00fce68ea86a38372bc7b64	11/19/2025 14:50:41	Tidal_Sensor	1763581829	46.5375	4.05	79.77
4	e00fce68ea86a38372bc7b64	11/19/2025 14:55:33	Tidal_Sensor	1763582129	51.92639	4.04	79.77
5	e00fce68ea86a38372bc7b64	11/19/2025 15:00:52	Tidal_Sensor	1763582429	52.06944	4.05	79.77
6	e00fce68ea86a38372bc7b64	11/19/2025 15:05:32	Tidal_Sensor	1763582730	52.1125	4.04	79.93
7	e00fce68ea86a38372bc7b64	11/19/2025 15:10:33	Tidal_Sensor	1763583029	52.07222	4.04	79.93
8	e00fce68ea86a38372bc7b64	11/19/2025 15:15:37	Tidal_Sensor	1763583329	52.04167	4.04	80.09
9	e00fce68ea86a38372bc7b64	11/19/2025 15:20:33	Tidal_Sensor	1763583630	52.25278	4.04	80.09
10	e00fce68ea86a38372bc7b64	11/19/2025 15:25:34	Tidal_Sensor	1763583930	51.45833	4.03	80.09
11	e00fce68ea86a38372bc7b64	11/19/2025 15:30:35	Tidal_Sensor	1763584229	149.28889	4.04	80.09
12	e00fce68ea86a38372bc7b64	11/19/2025 15:35:39	Tidal_Sensor	1763584529	67.34306	4.04	80.26
13	e00fce68ea86a38372bc7b64	11/19/2025 15:40:39	Tidal_Sensor	1763584829	66.48055	4.05	80.26
14	e00fce68ea86a38372bc7b64	11/19/2025 15:45:33	Tidal_Sensor	1763585129	66.6875	4.04	80.26
15	e00fce68ea86a38372bc7b64	11/19/2025 15:50:35	Tidal_Sensor	1763585430	66.97222	4.05	80.26
16	e00fce68ea86a38372bc7b64	11/19/2025 15:55:35	Tidal_Sensor	1763585729	67.35833	4.04	80.42
17	e00fce68ea86a38372bc7b64	11/19/2025 16:00:34	Tidal_Sensor	1763586030	67.15556	4.04	80.42
18	e00fce68ea86a38372bc7b64	11/19/2025 16:05:31	Tidal_Sensor	1763586329	67.24583	4.04	80.42
19	e00fce68ea86a38372bc7b64	11/19/2025 16:10:37	Tidal_Sensor	1763586630	67.55833	4.04	80.42
20	e00fce68ea86a38372bc7b64	11/19/2025 16:15:35	Tidal_Sensor	1763586930	68.12222	4.05	80.59
21	e00fce68ea86a38372bc7b64	11/19/2025 16:20:32	Tidal_Sensor	1763587230	67.78194	4.04	80.59
22	e00fce68ea86a38372bc7b64	11/19/2025 16:25:34	Tidal_Sensor	1763587530	67.425	4.04	80.59
23	e00fce68ea86a38372bc7b64	11/19/2025 16:30:39	Tidal_Sensor	1763587829	67.66805	4.05	80.59
24	e00fce68ea86a38372bc7b64	11/19/2025 16:35:41	Tidal_Sensor	1763588130	67.875	4.04	80.59
25	e00fce68ea86a38372bc7b64	11/19/2025 16:40:34	Tidal_Sensor	1763588429	71.09444	4.04	80.75
26	e00fce68ea86a38372bc7b64	11/19/2025 16:45:38	Tidal_Sensor	1763588729	68.16528	4.04	80.75
27	e00fce68ea86a38372bc7b64	11/19/2025 16:50:34	Tidal_Sensor	1763589029	67.84167	4.04	80.75
28	e00fce68ea86a38372bc7b64	11/19/2025 16:55:34	Tidal_Sensor	1763589329	68.10973	4.04	80.75
29	e00fce68ea86a38372bc7b64	11/19/2025 17:00:33	Tidal_Sensor	1763589630	67.81528	4.04	80.75
30	e00fce68ea86a38372bc7b64	11/19/2025 17:05:31	Tidal_Sensor	1763589929	68.40417	4.04	80.75
	e00fce68ea86a38372bc7b64	11/19/2025 17:10:36	Tidal_Sensor	1763590229	67.83334	4.03	80.91
	e00fce68ea86a38372bc7b64	11/19/2025 17:15:34	Tidal_Sensor	1763590530	68.25139	4.03	80.91

To make the data accessible and easier to analyze long-term, we integrated the Boron with the Particle Cloud and connected it to a live Google Sheet. Using Particle's built-in integrations along with Google Apps Script, the sensor's data is automatically published and updated in real time for remote monitoring and analysis

Particle Cloud - Showing the Boron and logging occurrences

Sandbox

Boron Boron 38848

sb2268@cornell.edu

Sandbox > Boron > Devices > View device

General information

Name: none

ID: e00fce68ea86a38372bc7b64

Owner: sb2268@cornell.edu

ICCID: 8988307000021704661

IMEI: 351457831036535

Serial number: P044AD3097F6J34

State

Status: Online

Last heard: Nov 18th 2025, 11:01 am

Last handshake: Nov 18th 2025, 10:35 am

Device OS: 6.2.1

Firmware: none

Groups: [+ Add groups](#)

[Signal](#) [Ping](#) [Edit](#)

mapbox [Open in maps](#)

Events Vitals SIM card

Search for events

Advanced

NAME	DATA	DEVICE	PUBLISHED AT
spark/status	offline	e00fce68ea86a38372bc7b64	11/18/25 at 11:47:16 am
waterLevel	946684810.3054722.3.49.0.00	e00fce68ea86a38372bc7b64	11/18/25 at 10:43:55 am
particle/device/updates/pending	false	e00fce68ea86a38372bc7b64	11/18/25 at 10:43:55 am
particle/device/updates/forced	false	e00fce68ea86a38372bc7b64	11/18/25 at 10:43:55 am
particle/device/updates/enabled	true	e00fce68ea86a38372bc7b64	11/18/25 at 10:43:54 am
spark/device/last_reset	pin_reset	e00fce68ea86a38372bc7b64	11/18/25 at 10:43:54 am
spark/status	online	e00fce68ea86a38372bc7b64	11/18/25 at 10:43:54 am
spark/device/diagnostics/update	{ "service": "device", "status": "unreac...	e00fce68ea86a38372bc7b64	11/18/25 at 10:43:47 am

Select an event to see more details about it

Last vitals

Nov 17th, 2025, 02:01PM

65% cellular signal strength

38% cellular signal quality

Operator: AT&T Wireless Inc.

Access technology: LTE

Cell Global Identity: 310-410-2346-28693265

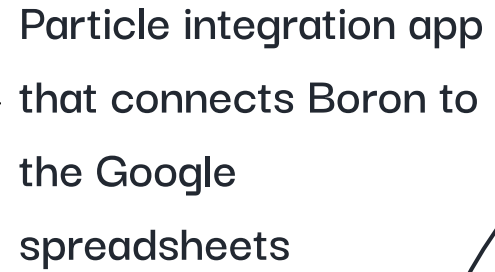
693ms round-trip time

75kB of 163.2kB of RAM used

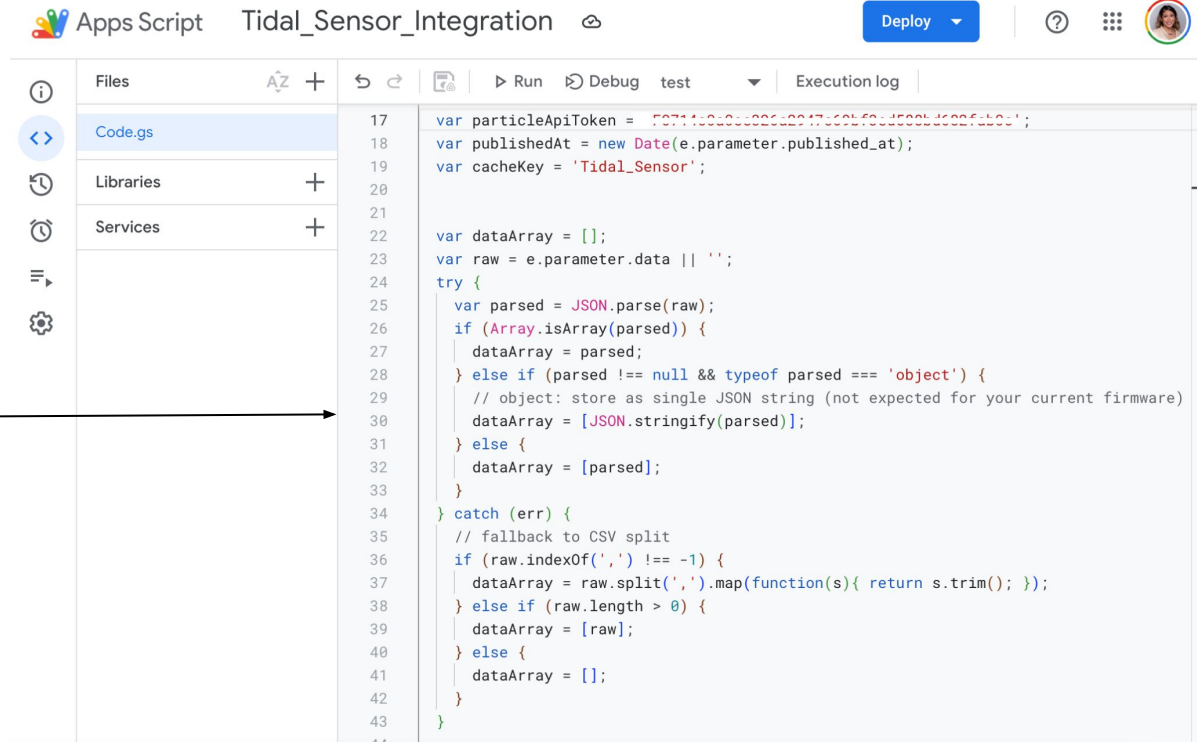
0 cloud disconnect events

0 rate-limited publishes

[Download history \(docs\)](#)



Apps Script that uses Particle Cloud API key to parse data, return it as a JSON, and use it in this script. (Apps Script is coded in python)



Data Analysis



After confirming the device was functioning properly and streaming data to Google Sheets, we adjusted the system to record data every 4 hours. This allowed us to visualize changes over time while conserving power.



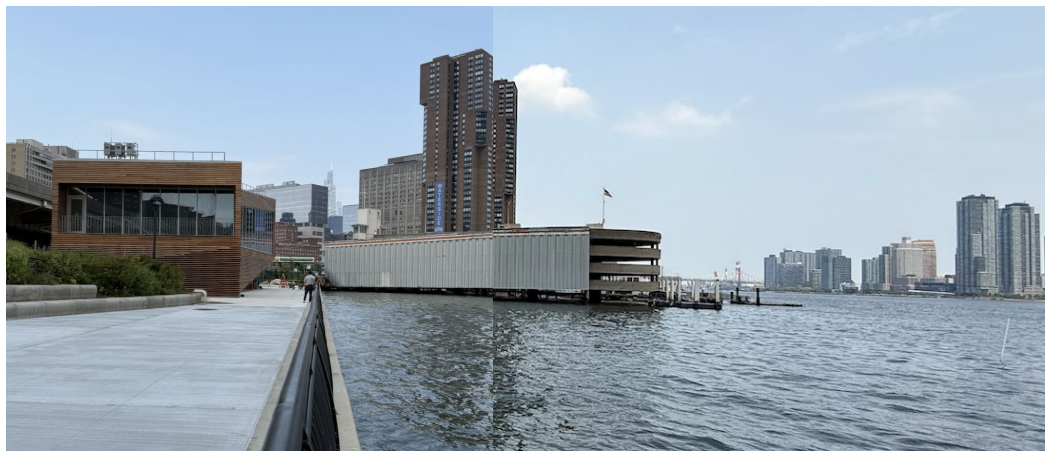
One important aspect to note is the frequency the data was being recorded



This will help visualize tidal patterns or detect outliers in readings

Deployment

The completed tidal sensor will be installed at the Stuy Cove Park along the East River, where it could directly monitor changes in water level near the seawall. They will use it to monitor the tide and boat wakes that sometimes rise above the rails, causing splash over into the park. Overall, this gives the park a simple way to track and respond to rising tides



Thank You

Thank you for following along with the development of our open-source tidal sensor. From assembly to data integration, this project highlighted the challenges and possibilities of real-world environmental monitoring. We appreciate the support of Stuy Cove Park, Phil Bresnahan, and everyone involved in making this work possible.